Prompt Engineering

In-context learning, clear instructions, prompt experiments, and layered defenses

Treat prompts as versioned interfaces. State the task clearly, supply the information needed, test variations systematically, and never rely on prompt wording alone to enforce security boundaries.

Source: Chip Huyen, AI Engineering: Building Applications with Foundation Models, first edition (2025). Source: book pp. 211–252 / PDF p. 235. These are original study notes, not verbatim chapter reproductions. Tool names, prices, model comparisons, and research findings reflect the book's publication period unless stated otherwise.

01 Introduction to Prompting

Source: book pp. 212–220 / PDF p. 236

In-context learning: zero-shot and few-shot

Prompt-based adaptation changes inputs, not model parameters. **Zero-shot** gives no demonstrations; **few-shot** supplies examples showing the desired mapping. More examples can help coverage but consume tokens, introduce bias, or distract the model.

Choose examples for relevance, consistency, and useful diversity. In-context learning can supply new information beyond training, but it does not persistently update weights. A model must already have enough capability and instruction-following ability to use the prompt; no phrasing trick guarantees missing skills.

System/user messages and chat templates

System messages typically define persistent application instructions; user messages contain the current request. Other roles carry prior assistant outputs and tool results. The model sees their serialized representation according to a specific chat template.

Use the exact tokenizer/template for the model version. Duplicated special tokens, wrong separators, missing role boundaries, and double-applied templates can quietly degrade behavior. When replacing models, retest the entire prompt construction path. Role labels help express instruction hierarchy but do not alone create secure isolation.

Context length versus context efficiency

A large advertised window describes how much the model accepts, not how reliably it uses all of it. Context efficiency asks whether it can find and combine relevant information wherever it appears. Needle-in-a-haystack and RULER-style tests probe long-context behavior, but simple needle retrieval does not establish multi-hop reasoning.

Test different lengths, evidence positions, distractors, and cross-document requirements. Longer prompts raise processing costs and can introduce conflicting information. Include the most useful context rather than filling the window merely because it exists.

---

02 Prompt Engineering Best Practices

Source: book pp. 220-235 / PDF p. 244

Write clear and explicit instructions

State the task, audience, required information, constraints, and output format without ambiguity. A persona can establish perspective but is not a substitute for criteria. Show representative examples when prose alone leaves room for interpretation.

Make compatible instructions; avoid asking simultaneously for extreme brevity and exhaustive explanation. Specify valid labels, schemas, refusal/uncertainty behavior, and whether extra commentary is allowed. Compact examples can reduce cost if evaluation confirms they teach the same behavior.

Provide sufficient context

Give facts the model needs, or provide authorized retrieval tools. Distinguish quoted source data from instructions and make provenance visible. Requests to use only supplied context should define what to do if evidence is missing or contradictory.

Prompting a model not to use outside knowledge is not a guarantee. For sensitive tasks, validate support for important claims or use constrained/extractive workflows. Missing context is often a retrieval/data problem rather than an instruction-wording problem.

Break complex tasks into simpler subtasks

Separate intent classification, evidence collection, transformation, and answer generation when each needs different instructions or checks. Route to task-specific prompts instead of packing every possible workflow into one prompt. Parallelize independent steps where appropriate.

Benefits include modular testing, better debugging, and use of cheaper specialized models. Costs include more calls, intermediate failures, and higher end-to-end latency. Evaluate the full pipeline: a stronger intermediate result does not guarantee a better final answer.

Give the model time to think

The book discusses chain-of-thought examples, task decomposition, and reasoning before an answer. These can improve multi-step tasks in some models, but their benefit depends on model training, task difficulty, and cost.

Use worked examples, explicit intermediate checks, or an appropriate reasoning model. Do not treat a convincing explanation as proof, and do not assume hidden/internal reasoning must be exposed to the user. Verify results independently when possible; longer output creates additional latency and opportunities for error.

Iterate and evaluate tools

Run prompt experiments on fixed evaluation data, track model/version/sampling, and compare quality, cost, latency, and robustness. Try meaningful perturbations, not just one hand-picked happy path. Read model guidance and learn its behavior before adopting elaborate abstractions.

Prompt optimizers can generate candidates or mutations and select better variants. Their gains depend on evaluation quality and can overfit the development set. Framework defaults can contain typos, wrong templates, or unannounced changes. Inspect the final prompt sent to the model.

Organize and version prompts

Separate reusable prompt definitions from business logic; name and document their purpose, model/template dependencies, schema, examples, and evaluation history. Store prompts as code or structured files so subject-matter experts can review them.

A versioned prompt catalog lets applications pin different versions; shared files that change automatically can force every consumer to migrate at once. Log the effective prompt and version, and review prompt changes like interface changes. Preserve rollback paths and avoid embedding secrets.

---

03 Defensive Prompt Engineering

Source: book pp. 235-252 / PDF p. 259

Threat model and proprietary prompts

Risks include prompt disclosure, unauthorized information access, harmful generated content, unexpected code/tool execution, and manipulation of application decisions. Proprietary prompt secrecy is not a reliable security boundary or durable product moat.

Reverse prompt engineering attempts to infer or extract hidden instructions. Even if the system prompt is exposed, the application should not reveal credentials or bypass permissions. Keep secrets out of model-visible context and enforce access through trusted application components.

Jailbreaking and prompt injection

Jailbreaking aims to bypass model restrictions; prompt injection attempts to redirect an application through model-visible instructions. Direct malicious requests and indirect instructions embedded in retrieved pages, emails, files, or tool outputs have different entry points.

Indirect injection is especially important because ordinary legitimate tasks can bring attacker-controlled content into context. A document being summarized is evidence, not an authorized operator. Automated adversarial testing can uncover failures, but defenses must be assessed across realistic tool workflows and languages.

Information extraction, memorization, and rights

Models can sometimes regurgitate training data, sensitive personal information, or copyrighted material. The ability to generate text does not imply the information is authorized to disclose. Memorization, extraction risk, and ordinary public knowledge are different phenomena.

Review training/source rights, reduce unnecessary sensitive context, limit retrieval by access permissions, and inspect outputs. No output detector proves the absence of private or copyrighted material. Provider terms and dataset licenses deserve explicit review before reusing outputs for training or commercial products.

Defenses at model, prompt, and system levels

- **Model:** adversarial training and instruction-hierarchy training to distinguish trusted directions from untrusted data.
- **Prompt:** explicit boundaries, clearly delimited inputs, repeated task reminders where useful, and safe handling of unsupported requests.
- **System:** authentication, least-privilege tools, schema/argument validation, sandboxed execution, allowlists, rate limits, anomaly detection, output checks, and human approval for consequential writes.

Measure refusal false positives as well as attack success. No single guardrail is foolproof. Test layers together and assume untrusted retrieved content will eventually contain adversarial instructions.

Revision Check

1. When do more demonstrations hurt?
2. How would you verify the correct chat template?
3. What does a large context window fail to guarantee?
4. When is decomposition worth its added latency?
5. Which security decisions must live outside the prompt?

Use explicit instructions and evidence, then earn confidence through tests. Prompt quality is important, but permissions and safety must also be enforced outside

generation.

Book-based notes, separate from the earlier transcript-based notes. Coverage is indexed by section in coverage.md. Source links open the supplied book at the relevant PDF page. Detailed examples, citations, and complete experimental evidence remain in the book.